

Secure Data Sharing in Cloud Using Attribute-Based Encryption (ABE)

A Project Report submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (Cyber Security)

Submitted by

**Patan Ahammad Sahid (BU22CSEN0400031)
Mohammed Mehraj Ahmed (BU22CSEN0400125)
Ranjith Gowda B M (BU22CSEN0400258)**

Under the esteemed guidance of

**Ms. Nida Kousar G
Assistant Professor**



GITAM
DEEMED TO BE UNIVERSITY

**Department of Artificial Intelligence and Data Science
GITAM School of CSE, GITAM (Deemed to be University)
Bengaluru
2026**

Secure Data Sharing in Cloud Using Attribute-Based Encryption (ABE)

A Capstone Project Report submitted in partial fulfilment of the requirements for the award of the degree of,

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (Cyber Security)

Submitted by:

**Patan Ahammad Sahid
Mohammed Mehraj Ahmed
Ranjith Gowda B M**

**BU22CSEN0400031
BU22CSNE0400125
BU21CSEN0400258**

Under the guidance of
Ms. Nida Kousar G
Assistant Professor



**Department of Artificial Intelligence and Data Science,
GITAM SCHOOL OF TECHNOLOGY
GANDHI INSTITUTE OF TECHNOLOGY AND MANAGEMENT
(Deemed to be University)
Bengaluru Campus**

March 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

GITAM SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

GITAM

(Deemed to be University)



DECLARATION

We, hereby declare that the project report entitled “**Secure Data Sharing in Cloud Using Attribute Based Encryption (ABE)**” is an original work done in the **Department of Artificial Intelligence and Data Science, GITAM School of Computer Science and Engineering, GITAM (Deemed to be University), Bengaluru** submitted in partial fulfilment of the requirements for the award of the degree of **B.Tech.** in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree.

Date:

Registration No(s).	Name(s)
BU22CSEN0400031	Patan Ahammad Sahid
BU22CSEN0400125	Mohammed Mehraj Ahmed
BU22CSEN0400258	Ranjith Gowda B M

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

GITAM SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

GITAM

(Deemed to be University)



CERTIFICATE

This is to certify that the project report entitled “**Secure Data Sharing in Cloud Using Attribute Based Encryption (ABE)**” is a Bonafide record of work carried out by **Patan Ahammad Sahid (BU22CSEN0400031), Mohammed Mehraj Ahmed (BU22CSEN0400125), Ranjith Gowda B M (BU22CSEN0400258)** submitted in partial fulfilment of requirement for the award of degree of **Bachelor of Technology in Computer Science and Engineering.**

Project Guide

Head of the Department

Ms. Nida Kousar G

Assistant Professor

Dr. Prof. A. Vadivel

HOD, Dept of AI & DS

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose consistent guidance and encouragement crowned our efforts with success.

We consider it our privilege to express our gratitude to all those who guided us in the completion of the project.

We express our gratitude to **Director Prof. Y. Vamsidhar** for having provided us with the golden opportunity to undertake this project work in their esteemed organization.

We sincerely thank **Dr. Prof. A Vadivel, HOD**, Department of Artificial Intelligence and Data Science, Gandhi Institute of Technology and Management, Bengaluru for the immense support given to us.

We express our gratitude to our project guide **Ms. Nida Kousar G, Assistant Professor**, Department of Artificial Intelligence and Data Science, Gandhi Institute of Technology and Management, Bengaluru, for their support, guidance, and suggestions throughout the project work.

Student Name's

Registration No.

PATAN AHAMMAD SAHID

BU22CSEN0400031

MOHAMMED MEHRAJ AHMED

BU22CSEN0400125

RANJITH GOWDA B M

BU22CSEN0400258

ABSTRACT

Sharing files safely, so only the right people can see them, can be tricky. This project is a secure way to do it, using tech like blockchain, encryption, and a simple website. Normal file sharing trusts central systems, which can be hard to check. This system uses blockchain to control who gets access and strong encryption to protect the files. It's built to be easy to use, keep your info private, and show who did what. Basically, the system scrambles files with AES-256 for safety. Then, it locks the key with CP-ABE, linking access to things about you (like your job title) instead of just your name. The scrambled key and access info are kept safe, and the file itself can be stored on your computer or in the cloud. This way, no one can read the data without the right keys, even if the storage is hacked.

To control who sees what, the system uses a smart contract on a test Ethereum network. When you ask for access, it's checked and recorded. If approved, it's put on the blockchain, creating a permanent record. This shows everything without revealing private info. The backend, built with FastAPI, handles uploads, logins, requests, and downloads. It also logs everything, giving admins a clear view of how data is being used. The frontend, made with React, is designed to be easy to use. You can ask for permission, see files you can access, connect your wallet, and download approved files. Tests confirm that everything works together smoothly. The system checks user logins, wallet connections, uploads, requests, approvals, and secure downloads. It's a simple process: log in, connect a wallet, request access, and download files once approved. Each step is secure and trackable.

In short, this project is a complete system for sharing data safely. It combines security tech with blockchain to make sure everything is above board. It proves that strong security doesn't have to be hard to use, giving users control, admins transparency, and verifiable access decisions. It's a good fit for fields where trust and privacy are important, like healthcare or law.

TABLE OF CONTENTS

TITLE	PAGE NO.
DECLARATION	II
CERTIFICATE	III
ACKNOWLEDGEMENT	IV
ABSTRACT	V
1. INTRODUCTION	1
1.1 Background	2
1.2 Purpose	3
1.3 Scope	3
2. LITERATURE SURVEY	5
2.1 Literature Survey	7
2.2 Key Findings	8
2.3 Research Gap	8
3. PROBLEM FORMULATION AND OBJECTIVES	9
3.1 Problem Statement	9
3.2 Objectives	10
3.3 Real-Life Use Cases	10
4. REQUIREMENT ANALYSIS	12
4.1 Software Requirements	12
4.1.1 Functional Requirement	12
4.1.2 Non-Functional Requirement	12
4.2 Hardware Requirements	12

5. FLOWCHART OF THE SYSTEM	14
6. SYSTEM ARCHITECTURE	16
7. METHODOLOGY	18
8. TOOLS AND TECHNIQUES USED	20
8.1 Tools and Software	20
8.2 Techniques Used	20
8.3 Algorithms Used	21
8.4 Development Environment	21
9. IMPLEMENTATION	23
10. EVALUATION AND DISCUSSION	27
11. RESULTS AND ANALYSIS	29
12. CONCLUSION	33
13. FUTURE WORK	34
14. CO-PO-PSO MAPPING	35
15. REFERENCES	43

LIST OF FIGURES

Figure	Title	Page No.
5.1	Secure data sharing workflow	15
6.1	System architecture	17
11.1	Admin login and user dashboard	29
11.2	User Registration by Administrator	29
11.3	Secure file upload with access policy	30
11.4	Encrypted file storage confirmation	30
11.5	User login and file access request	31
11.6	Access policy verification result	31
11.7	Blockchain-Based Multi-Authority Approval	32
11.8	Secure file download after approval	32

LIST OF TABLES

Table No.	Title	Page No.
9.1	Libraries used and their purpose for User Authentication	23
9.2	Libraries used and their purpose for file encryption and storage	24
9.3	Libraries used and their purpose for attribute-based access control	25
9.4	Libraries Used and Their Purpose for Blockchain Approval	26

CHAPTER 1

INTRODUCTION

This project tackles a common issue in data sharing these days: how to share sensitive info while keeping it private, controlling who sees what, and tracking who accessed it. Normal file-sharing setups usually have central control and trust built-in, which makes it hard to check things and ups the chance of someone getting in who shouldn't. Our system aims to lower that chance by using strong encryption, access control based on policies, and blockchain checks. When you upload a file, it's encrypted with AES-256, so nobody can read it without the right key. Then, the encryption key is protected by a method that lets you set access rules based on things like role, department, or permission level. This gives you options for who can decrypt a file without needing a list of keys for each person. To ensure access decisions can be checked and aren't changed, we've put in a smart contract on an Ethereum test network. People ask for access through a wallet, and a tool verifies if the request fits the policy. Approved decisions are put on the blockchain, creating a clear audit trail. This keeps everyone responsible without showing sensitive data on the blockchain.

The backend, created with FastAPI, has safe spots for signing up, uploading files, asking for access, oracle checks, and downloads. The frontend, made in React, focuses on being easy to use, making wallet, finding files, and sending requests easy for people. A full logging system gets key actions and transaction details, helping with rules and checks. Full tests check the process from signing up to downloading files safely, showing that the system works as a solid, dependable solution. All in all, this project is a secure data-sharing platform that balances keeping things secret, controlling access, and making it easy for users—making it good for real-world where data privacy and responsibility are super important.

1.1. Background

The amount of private info shared inside companies has shot up thanks to digital changes. Stuff like financial reports, worker info, audit reports, and plans moves around between teams, managers, and people outside the company all the time. Normal ways to control who sees what can block access at the program level, but they usually depend on central

servers and admins. This can cause problems like a single point of failure, not being able to track access well, and trouble making sure only those who need to know get access. So, keeping data safe when sharing it needs a few layers of protection. First, keep things secret, even if the storage is hacked. Only those with special keys should be able to read the files. Second, access control should be specific and consider the situation. Rules like letting only people with certain roles and departments get the key to decrypt it. Third, make sure permissions are checked and can't be easily abused. This is super important when approvals need to be accountable and involve more than one person.

This project aims to solve these problems by:

- Making sure files are secret using a way of encrypting them. Files get encrypted to a format that has a special code, followed by encrypted text. This way, even if someone breaks into the storage, the files stay protected.
- Controlling access using a token system. Policies use attribute tokens (like role:admin, dept:IT) with logic (AND/OR). These policies say if a user can get the key needed to unencrypt.
- Making authorization decentralized using a blockchain approval system. A digital contract tracks approvals from different authorities, and access is given only when enough people agree (like 4 out of 7 authorities). This adds multi-person control and a way to track approvals on a test Ethereum network.
- The system uses a FastAPI setup and React. Files are saved as encrypted stuff in MongoDB GridFS, and other info is stored in a simple SQLite database. The whole plan shows how encryption and approval can be put together into a safe sharing system for testing and demos.

1.2. Purpose

Okay, so this project is all about building a safe way to share data. It's meant to keep prying eyes away from private files, but still let the right people get to them through a system that can be checked.

Here's what it's supposed to do:

- 1) Keep files totally secret by encrypting them with a tough code (AES-256-CBC) before they're put away.

2) Make sure only folks with the right qualifications can decrypt the files, based on who they are and what they do.

3) When someone wants to get to really private stuff, it needs to be okayed by several people using a blockchain system. That way, one person can't just give access without enough others agreeing (like needing 4 out of 7 people to say yes).

4) It'll have an easy website where admins can add users, upload files with rules about who can see them, ask for access, pretend to be approvers (for testing), and grab/unscramble files safely.

5) Show how you can check who gets access to what by looking at the blockchain. That way, everyone can clearly see who approved what, making sure things are fair.

1.3. Scope

This project is all about building a working model for sharing files securely. It uses encryption and a decentralized approval system. Here's what's included:

A) System Parts:

- Backend: REST APIs that handle logins, file uploads, listings, access requests, approval checks, and secure downloads.
- Frontend: A web app for logging in, managing users (admin stuff), uploading files with set rules, starting the access approval process, and downloading/decrypting files after approval.
- Smart Contract: A KeyAuthority contract that keeps track of who can approve things and records votes for each key. Access is granted when enough approvals are in Storage: Encrypted files are kept as blobs in MongoDB GridFS. (Info like users, files, policies, and recovery codes is stored in an SQLite database).

B) Security Features:

- Files are encrypted when stored using AES-256-CBC with random starting values.
- Access control is policy-based, using attribute tokens and simple logic (AND/OR).
- This makes sure access isn't just about roles but also other details.

- Approval needs multiple authorities using a blockchain system (4 out of 7 needed).
- Admin stuff like adding users is controlled. Self-registration is off, passwords are safe with hashing, and there's a recovery code thing.

C) Where it's meant to run:

- A local setup for testing:
- Ganache as a fake blockchain.
- Smart contract set up through Remix or a script.
- Backend runs on a local server, and the frontend runs on a local server too.
- Encrypted files are stored in MongoDB GridFS (using environment variables to set it up).

CHAPTER 2

LITERATURE SURVEY

S.No	Paper Details	Summary	Techniques/ Algorithms Used	Research Gap
1.	Title: An SKP-ABE Scheme for Secure and Efficient Data Sharing in Cloud Environments Author: Y.-W. Hwang et al., Year: 2022.	Proposes Searchable Key-Policy ABE (SKP-ABE) that enables fast ciphertext search using aggregated attribute indices and outputs constant-size ciphertexts.	KP-ABE + searchable encryption techniques; attribute aggregation / hashing for index	High encryption overload due to aggregation; lacks integrity verification and real – world implementation results.
2.	Title: Secure Data Sharing in the Cloud Authors: D. Thilakanathan et al., Chapter (Springer), Year: 2014.	Secure cloud data sharing techniques (ABE, PRE, re-encryption, revocation). Reviews key management, revocation approaches.	CP-ABE, KP-ABE, proxy re-encryption (PRE), clock-based PRE schemes, re-encryption delegation.	efficient large-scale revocation, accountability/auditing, enforcing usage/policy after sharing (DRM-style controls),

S.No.	Paper Details	Summary	Techniques/ Algorithms Used	Research Gap
3.	Title: FP2E: Flexible, Effective and Privacy-Preserving Cloud Data Sharing Scheme. Author: Imad El Ghoubach et al. Year: 2016	FP2E: Extends P2E with threshold-gate access structures, offering more flexibility and lower key generation/encryption overhead than CP-ABE and P2E.	CP-ABE with LSSS/threshold-gate access structure3333s, AES for encryption.	Limitations & Future Work: Lacks delegation and data-integrity checks; future work includes adding these features and testing on realistic workloads.
4.	Title: A Comprehensive Survey of Attribute-Based Encryption Techniques in Cloud Computing Authors: R. Fatima & M. Naveeduddin Year: 2025.	Recent survey that categorizes ABE variants (CP-ABE, KP-ABE, HABE, MA-ABE), discusses lightweight ABE for IoT, blockchain-assisted ABE, and post-quantum directions.	Meta-analysis of many ABE schemes; taxonomy; comparison tables; discussion of post-quantum ABE, hierarchical and multi-authority schemes and tradeoffs.	Highlights gaps: real post-quantum ABE deployments, standards for attribute management, integration with decentralized trust (blockchain) at scale
5.	Title: Lattice-based ABE with multi-ciphertext equality test in cloud computing Authors: M. Zhao et al., <i>J. King Saud Univ. — Comp. & Info. Sci.</i> , Year: 2025.	Introduces a quantum-resistant CP-ABEMET (integrates CP-ABE with multi-ciphertext equality test): enables a single equality test over many ciphertexts (using Shamir-style shares + polynomial hashing)	Lattice (LWE) constructions; Shamir (η, β) threshold secret sharing; CP-ABE for policy encryption; techniques to produce equality tokens	Tradeoffs: higher storage and parameter setting overhead; open problems include achieving CCA2, reducing storage/communication cost,

2.1. Literature Survey:

Keeping data safe when you share it is really about making sure only the right people can see it, even if the places where you keep the data or the ways you send it aren't totally safe. Usual ways of protecting data, such as access controls, mostly control who gets to see things within an app. But, if your data is on something like cloud storage, then there are new ways for bad guys to get in, like if someone sets up the storage wrong, steals passwords, sells out the company or doesn't check things carefully enough.

From what we saw in our project, you can build a secure sharing system by putting together three small ideas:

- Keep things secret by Encrypting the files before they are stored (we used AES-256-CBC for this).
- Control who can see what by using rules based on things like job title, department to decide if someone can decrypt a file.
- Make sure more than one person has to agree before someone can decrypt a file; for example, you might need 4 out of 7 approvers. We recorded these approvals on a blockchain.

This part looks at what other people have done in areas like encrypting data based on who should see it, how to keep encryption keys safe, how to use blockchains to control access, and how to keep cloud storage secure. It ends by sizing up the different ways to do things and figuring out what our project is improving.

Relevance to the Project:

- The system uses CP-ABE-like rules and checks for things like roles and clearances. CP-ABE might not work everywhere, but this project shows the main idea: rules decide if someone can decrypt something, not just who they are.

2.2. Key Findings from Literature Survey

- To keep data safe in the cloud, you really need encryption (AES).
- Normal Role-Based Access Control (RBAC) can only do so much. Attribute-Based Access Control (ABAC/CP-ABE) lets you control who sees what much

better.

- CP-ABE gives great security with its policies, but it can be tricky to set up.
- If you only have one place controlling who gets access, that spot can fail or someone on the inside could mess things up.
- Blockchain makes things easier to see, check, and keeps approvals from being messed with.
- Using things like 4-of-7 models, where it takes 4 out of 7 people to approve something, cuts down on the chance of someone abusing their authority.
- When saving stuff to the cloud, make sure encryption is built in from the start.
- How you handle your keys is just as important as encrypting the data itself.
- One thing by itself won't cut it, so a mix of encryption, attribute policies, and approvals that aren't all in one place is safer and works better.

2.3. Research Gap

- Current access systems that are centralized depend on one authority that is trusted, this creates points of failure and risks from the inside.
- CP-ABE gives crypto enforcement that's fine-grained, but it is hard to use and complex in the real world.
- Access control that is based on blockchain makes auditing better, but it does not give data confidentiality that is built-in.
- A lot of systems focus just on encryption or authorization, but not together in one way.
- There aren't many prototype systems out there that combine encryption that's efficient, attribute-based eligibility, and approval that is decentralized/multi authority into one design.

CHAPTER 3

PROBLEM FORMULATION AND OBJECTIVES

3.1. Problem Statement

Companies share private files inside their offices and need to make sure only certain people can see them.

The system has to:

- Keep things secret – Even if someone breaks into the system, files stay encrypted.
- Give precise control – Access depends on things like your job, team, or security level.
- Need multiple approvals – No one person can just give access.
- Track approvals – Easy to check approvals and make sure no one messed with them.

Current System

A main server checks who can see what and then sends the files.

Problems:

- If the server goes down, everything stops
- Risk of someone on the inside doing wrong
- Hard to track who approved what
- Limited control over who sees what

Proposed System

A three-part security setup:

- Encryption – Files are encrypted with AES-256-CBC before they go to the cloud.

- Rules – Access is based on roles (using AND/OR conditions).
- Blockchain – Needs 4 out of 7 authorities to approve access.
- Admins control who gets access and user info is protected.

3.2. Objectives

- Ensure confidentiality of shared files Encrypt every uploaded file using AES-256 before storing, so data remains protected at rest and during transfer.
- Provide fine-grained, attribute-based access control Define “who can access” each file using attribute policies (AND/OR rules) based on user attributes like role, department (dept), and clearance.
- Enforce decentralized approval before decryption Require a blockchain-based threshold approval (4-of-7 authorities) for each access request so no single party can grant access alone.

3.3. Real-Life Use Cases

Banking & Finance

- Sharing private financial info between groups.
- Big money moves need a few people to say yes.
- Keeping customer info safe in the cloud.

Healthcare

- Doctors and staff sharing patient files.
- Access depends on your job (like doctor), team (cardiology), and security level.
- Important patient files need a few approvals.

Government & Defense

- Sharing top-secret papers safely.
- Access depends on your security clearance (secret, top secret, etc.).
- Key intel files need many approvals.

Big Companies

- HR sharing employee info with bosses.
- The money people sharing audit reports with the higher-ups.
- Making sure no one person can mess around with important access.

Research Centers & Schools

- Researchers sharing data safely.
- Stopping people from seeing work that's not public yet.
- Getting a couple of approvals to see important project files.

Law Firms

- Lawyers sharing case info with senior partners.
- Access limits depend on your job and if you're working on the case.
- Keeping track of who approved access.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1. Software Requirements

4.1.1. Functional Requirements

- Users need to log in with a secure username and password.
- Only admins can make and handle user accounts.
- The system needs to give users attributes like role, department, and clearance.
- Admins need to be able to upload files.
- All uploaded files need to be encrypted with AES-256 before saving.
- Encrypted files need to be securely stored in the cloud.
- Access to each file should be based on user attributes.
- Files can only be downloaded after policy checks and approval from multiple authorities.

4.1.2 Non-Functional Requirements

- The system has to keep data private by encrypting files when they're not being used.
- Passwords and recovery info must only be stored as hashed data.
- Even if the cloud storage is cracked, unauthorized access should be stopped.
- Access control should be reliable using blockchain-based approvals.
- Encryption and decryption need to happen fast enough.
- The system should still work even if there are network or service issues.
- It should be easy to use, with a simple interface.
- The system should be built so it can be maintained and expanded in the future

4.1.3 Hardware Requirements

- For the system to run well, you'll need a computer with an Intel Core i5 processor, or something similar.
- At least 8 GB of RAM is required to run the backend, frontend, blockchain services, and to handle encrypted files and system logs simultaneously.
- Make sure you have at least 100 GB of free space on your hard drive for the

software, encrypted files, and logs.

- The system needs a 64-bit architecture to work with today's tools and libraries.
- You'll need a stable internet to get to cloud storage and install packages.
- A regular keyboard and mouse are needed so you can use the system and handle admin stuff.
- You'll need a display monitor that's at least 1366x768 so you can see everything properly.
- If you want extra security for your keys, you can use optional hardware security options (TPM / secure storage).

CHAPTER 5

FLOWCHART OF THE SYSTEM

The system flowchart describes the complete secure data sharing process starting from system access and ending with secure file download. It explains how cryptographic confidentiality, attribute-based access control, and blockchain-based multi-authority authorization operate sequentially to ensure secure data sharing.

- **System Start:** When you first use the system, we get everything ready behind the scenes –setting up cloud storage, the database, and the blockchain stuff. This gets the encryption, rule checks, and approval steps all set.
- **User Login:** You log in with your username and password. The system checks if they match what we have on file. If they don't, you can't get in.
- **Admin Uploads:** If you're an admin, you can upload files. You'll set up who can see those files based on things like your job, department, and security level.
- **Encrypting & Saving Files:** The file gets encrypted using a tough method (AES-256) with a random key. The encrypted file is saved in the cloud, and things like access rules are kept in the database.
- **Requesting Files:** When someone wants a file, they ask for access. The system makes a special ID to follow the request through the approval steps
- **Checking the Rules:** The system sees if the person asking for the file actually meets the rules for accessing it. If they don't, they can't get it.
- **Verification with Blockchain:** If the person meets the requirements, the blockchain verification thing kicks in. Several people then have to approve the request – like if 4 out of 7 authorities say it's okay, then access is granted
- **Decrypt & Download:** If the policy is good and enough people approve, the system gets the encrypted file from the cloud, decrypts it, and lets the person download it safely
- **Logging Out:** When you're done, the session ends. Everything that happened – access requests, approvals, and other stuff – is written to a log for later review.

Secure Data Sharing — System Workflow Flowchart

AES confidentiality + Attribute policy check + 4-of-7 blockchain approvals + Secure download

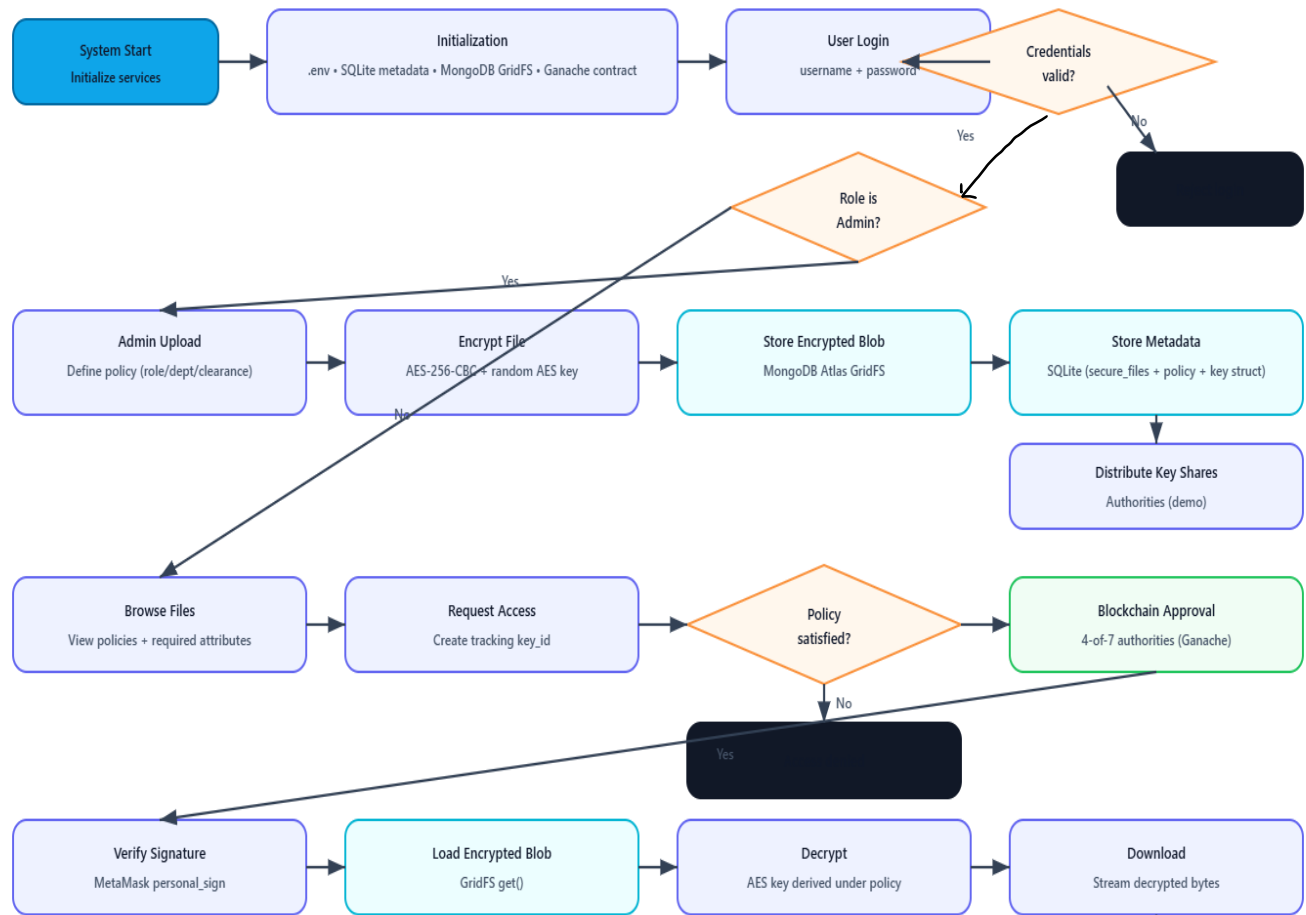


Figure 5.1: System flowchart illustrating secure data sharing workflow including user authentication, AES-256 file encryption, attribute-based access policy verification, blockchain-based multi-authority approval, and secure file download.

CHAPTER 6

SYSTEM ARCHITECTURE

The architecture of the proposed secure data sharing system integrates cryptographic protection, policy-based access control, decentralized authorization, and cloud storage to ensure confidentiality, controlled access, and auditability of sensitive files. The system is designed as a modular client–server architecture suitable for academic demonstration and controlled deployment environments.

- **User Interface:** Admins and other users use a website to get into the system. They can log in, manage users (if they're admins), upload files with rules about who can see them, ask for access, check if their requests are approved, and download files securely. The website talks to the behind-the-scenes stuff using standard web commands. The user interface serves as the primary interaction layer for authentication, file management, access requests, and secure downloads.
- **Login and User Management:** The system checks who you are using usernames and passwords. Passwords are kept safe in the database. Only admins can make new accounts, so we know who's on the system.
- **File Encryption:** Before a file is saved, it's encrypted using a strong method (AES-256) with a random key. The encrypted file is saved as a blob (a bunch of data) that includes this key. This means that even if someone gets into the cloud storage, they can't read the files.
- **Access Control:** Each file has rules about who can see it, based on things like their role and department. These rules use logic (like and and or). When someone asks to see a file, the system checks if they meet the rules before it does anything else
- **Decentralized Approval:** For really important files, more than one person needs to say it's okay to access them. The system uses a local test blockchain network to record these approvals using a smart contract. Access is given only when enough people approve (like 4 out of 7), so one person can't just let anyone in.

- **Cloud Storage:** The encrypted files are stored in MongoDB GridFS. Other info, like file names, rules, and encryption keys, is stored in a simple database.
- This keeps the files safe and easy to get to without putting plain files on the cloud.
- **File Retrieval and Decryption:** If you pass the policy check and blockchain approval, the system gets the encrypted file from cloud storage, decrypts it in memory, and sends the file to you securely. The decrypted file is never saved on the server.
- **Logging:** Every login, access request, approval, and file access is logged. Approvals done on the blockchain network leave behind a log that can't be changed, so you can always see who approved what.

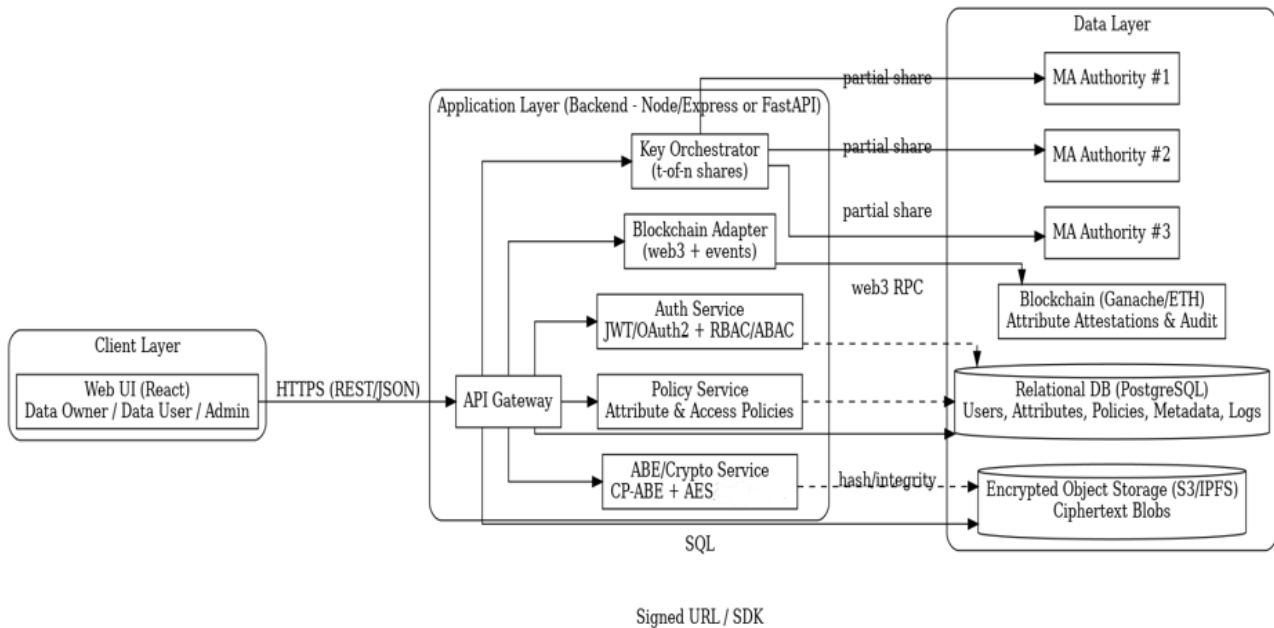


Figure 6.1: System architecture of the secure data sharing platform showing frontend interface, FastAPI backend services, AES-256 encryption module, attribute-based access control, blockchain-based multi-authority approval, cloud storage for encrypted files, and audit logging mechanism.

CHAPTER 7

METHODOLOGY

The proposed system follows a structured and modular methodology to design, implement, and evaluate a secure data sharing platform that ensures confidentiality, fine grained access control, and verifiable multi-authority authorization. The methodology emphasizes cryptographic protection of data, policy-based eligibility verification, and blockchain-backed approval to prevent unauthorized access.

1. **System Setup and Input Acquisition:** The first step involves setting up the development and execution environment. The backend services are configured using FastAPI, along with database initialization for user and file metadata storage. Cloud storage credentials are configured to enable encrypted file storage, and a local blockchain environment is initialized to support decentralized approval workflows.
2. **User Provisioning and Authentication Setup:** User accounts are created exclusively by an administrator to ensure controlled onboarding. User credentials are securely stored in hashed form in the database. During system usage, users authenticate using username and password verification before accessing any system functionality.
3. **File Upload and AES-Based Encryption:** Administrators upload files through the system interface. Before storage, each file is encrypted using AES-256 in CBC mode with a randomly generated initialization vector. The encrypted output is packaged as a secure blob, ensuring that file content remains confidential even if storage is compromised.
4. **Access Policy Definition and Storage:** During file upload, an attribute-based access policy is defined using user attributes such as role, department, and clearance. These policies are stored alongside file metadata in the database and are later used to determine user eligibility for decryption and access.
5. **Access Request and Policy Verification:** When a user requests access to a file, the system verifies whether the user's attributes satisfy the associated

access policy. If the policy conditions are not met, the access request is rejected, preventing unauthorized progression in the workflow.

6. **Blockchain-Based Multi-Authority Approval:** For eligible access requests, the system initiates a decentralized approval process using a blockchain smart contract. Multiple authority accounts independently approve the request, and access is granted only when the predefined threshold (4 out of 7 approvals) is reached. This ensures verifiable and tamper-resistant authorization.
7. **Secure File Decryption and Download:** After successful policy satisfaction and blockchain approval verification, the system retrieves the encrypted file from cloud storage. The file is decrypted in memory and securely streamed to the authorized user. Plaintext data is not stored persistently on the server.
8. **Audit Logging, Testing, and Validation:** All authentication events, access requests, approval actions, and file downloads are logged in the system for auditing purposes. The system is tested under multiple scenarios, including authorized access, policy violations, insufficient approvals, and configuration failures, to validate correctness, security, and reliability.

CHAPTER 8

TOOLS AND TECHNIQUES USED

8.1. Tools and Software

- **Python:** Python is used as the core programming language for implementing backend logic, encryption workflows, policy verification, and blockchain interaction due to its simplicity and strong library support.
- **FastAPI:** FastAPI is used to develop RESTful backend APIs for user authentication, file upload, access request handling, approval verification, and secure file download.
- **React:** React is used to build the frontend user interface, providing pages for login, admin user management, file upload with policy definition, access approval workflow, and file download.
- **SQLite Database:** SQLite is used as a lightweight database to store user credentials (hashed), user attributes, file metadata, access policies, and recovery code information.
- **MongoDB Atlas (GridFS):** MongoDB Atlas uses GridFS to keep encrypted files safe. It only stores the encrypted versions in the database, never the original files. When someone who is allowed to download a file does so, it's decrypted in memory only after the system confirms they have permission.
- **Ganache (Local Ethereum Blockchain):** To test the smart contract (the thing that manages the multi-person approval process), I used Ganache. It let me play around with a fake Ethereum blockchain on my computer.
- **Web3.py:** To get my app talking to the blockchain and interacting with that smart contract, I used Web3.py.
- **Diagram Tools (Draw.io / Lucidchart):** It helped us to draw system diagrams, flowcharts, and other visual stuff for docs.
- **Development Tools (VS Code, Git):** I used VS Code to write code and fix bugs, and Git to keep track of all the changes I made to the project.

8.2. Techniques Used

- **Symmetric File Encryption:** I encrypted the files using AES-256 before

storing them. This way, even if someone got into the storage, they wouldn't be able to read the files.

- **Attribute-Based Access Control (ABAC):** I set up access rules based on things **like** your role, department, and security clearance. These rules determined whether you were allowed to decrypt a file.
- **Policy-Based Eligibility Verification:** Before letting you access a file, the system checks whether your attributes match the access rules. It uses simple logic like AND and OR to see if you're allowed in.
- **Blockchain-Based Multi-Authority Approval:** I made it so that accessing a file requires approval from multiple people (4 out of 7, to be exact). This approval process is managed by a smart contract on the blockchain.
- **Separation of Data and Metadata:** The encrypted files are stored using MongoDB Atlas GridFS, while file metadata and access policies are stored in the database. This reduces the risk of everything being compromised at once.
- **Secure Key Handling (Envelope Encryption Concept):** The keys needed to decrypt files are protected by the access rules and are only released after your access is approved.
- **Audit and Traceability Mechanism:** Every access request, approval, and download is recorded. This helps with monitoring, accountability, and making sure everything is on the up-and-up.

8.3. Algorithms Used

- **AES-256-CBC Encryption Algorithm:** This is the algorithm I used to encrypt and decrypt the files.
- **Policy Satisfaction Algorithm:** This is how the system figures out whether your attributes match the access rules.
- **Blockchain Threshold Approval Algorithm:** Used to verify and record multi-authority approvals on the blockchain. Access to encrypted files is granted only when the predefined threshold (e.g., 4 out of 7 authorities) is satisfied.

8.4. Development Environment

- **Operating System:** I used Windows to build, test, and show off the project.
- **IDE / Code Editor:** VS Code was my go-to for writing, fixing, and

managing the code.

- **Blockchain Environment:** Ganache let me play with a local blockchain, so I could test the smart contract without spending real money on transaction fees.
- **Cloud Configuration:** The system was tested in a local development environment using MongoDB Atlas (GridFS) for encrypted file storage, SQLite for metadata, and a local Ethereum blockchain (Ganache) for testing multi-authority approval workflows.
- **Testing Environment:** I tested everything locally using fake users, access requests, and approval scenarios. This helped me make sure the encryption, access rules, and blockchain stuff were all working as expected.

CHAPTER 9

IMPLEMENTATION

1. Secure User Authentication and Access Control Module: The objective of this module is to ensure that only authorized users can access the secure data sharing platform. It verifies user identity using controlled authentication mechanisms and enforces administrative restrictions on sensitive operations such as user provisioning and file upload. Implementation Details

- **Programming Language:** Python

Table 9.1: Libraries Used and Their Purpose for User Authentication

LIBRARY	PURPOSE
FastAPI	Backend API development and request handling
SQLAlchemy	Database ORM for user and file records
SQLite	Storage of user credentials and attributes
bcrypt	Secure hashing and verification of passwords

- **User Data Handling:** We only store passwords and recovery codes in a scrambled form. Regular people can't just sign up; admins have to make accounts.
- **Authentication Logic:** When you log in, the system checks if your password matches the scrambled version we have stored. If it's right, we grab your info (like your role and department) to figure out what you're allowed to see and do.
- **Security Measures:** Unauthorized users are denied access immediately. Administrative actions such as user creation, file upload, and deletion are restricted to users with admin privileges.
- **Models / Algorithms Used:** We use attribute-based policy evaluation along with cryptographic verification. The system checks if a user's attributes

match the access rules set for a file using simple logic (AND / OR conditions).

System Implementation: The login and access control stuff is built right into the main system. When you log in, we check your identity using hashed info. We keep a record of all login attempts (good and bad) in the database. If you try to get in where you shouldn't, we block you right away. Only admins can do certain things.

2. Secure File Upload and Encryption Module: The objective of this module is to protect file confidentiality by encrypting sensitive data before storage and preventing plaintext exposure.

Implementation Details

- **Programming Language:** Python

Table 9.2 – Libraries Used and Their Purpose for File Encryption and Storage

LIBRARY	PURPOSE
cryptography	AES-256-CBC encryption and decryption
FastAPI	Multipart file upload handling
MongoDB Atlas (GridFS)	Storage of encrypted file blobs
SQLite	Storage of file metadata and policies

- **Encryption Process:** When an administrator uploads a file, the system generates a random AES-256 key and initialization vector. The file is encrypted using AES in CBC mode, producing an encrypted blob consisting of the IV followed by ciphertext.
- **Storage Strategy:** Only encrypted file blobs are stored in MongoDB using GridFS. Plaintext files are never written to disk or stored in the database.
- **Algorithms Used:** AES-256-CBC
- **System Implementation:** This module ensures encryption-by-design and protects data confidentiality even if storage resources are compromised.

2. Attribute-Based Access Policy: To control who can see files based on things like their role, department, and security clearance.

Implementation Details

- **Programming Language: Python**

Table 9.3 – Libraries Used and Their Purpose for Attribute-Based Access Control

LIBRARY	PURPOSE
Python Core	For understanding and using the access policies.
SQLAlchemy	Retrieval of user attributes and file policies
SQLite	Storage of access policies and metadata

- **Policy Definition:** When a file is uploaded, policies decide who can see it, using things like role, department, and clearance, combined with and/or rules.
- **Policy Verification:** When you try to open a file, the system checks if your attributes match the access policy. If they don't, access is denied.
- **Authorization Flow:** Only users who satisfy the access policy are allowed to proceed to the blockchain-based approval stage.
- **Algorithms Used:** Rule-based attribute matching and logical expression evaluation.
- **System Implementation:** This module is part of the process when you ask to see a file. It's like a gatekeeper that checks if you're eligible before you're allowed to get approval and decrypt the file. It gives detailed and flexible access control without showing the encrypted data.

3. Blockchain-Based Multi-Authority Approval Module: This module makes sure that getting access to a file requires agreement from several independent sources. This way, it's not just one person who decides.

Implementation Details

- **Programming Language: Python and Solidity**

Table 9.4 – Libraries Used and Their Purpose for Blockchain Approval

LIBRARY	PURPOSE
Web3.py	Talks to the blockchain smart contract
Ganache	Pretends to be an Ethereum blockchain for testing
Solidity	Sets up the smart contract for approvals

- **Getting Approvals:** When someone is eligible to ask for access, a special ID is made for their request. Multiple authorized accounts then give their approval through the blockchain smart contract.
- **Verification Logic:** The system checks the smart contract to see if enough approvals have been given (like 4 out of 7 authorities).
- **Security Enforcement:** Access is only given if the rules are met and the blockchain approval is confirmed.
- **System Implementation:** This module makes sure that no single person can approve access to a file on their own. Everything is recorded on the blockchain, so it's clear and easy to audit.
- **System Integration:** All the modules work together in one secure app for sharing data. User login manages who can use the policy evaluation, blockchain approval, and file decryption modules. Everything is logged and tracked in one place, so the system works as a single, secure way to share data.

CHAPTER 10

EVALUATION AND DISCUSSION

Evaluation: We tested our secure data sharing system to see how well it did with security, accuracy, reliability, and speed. We wanted to check if the system correctly encrypted files, used the right access rules, and got approval from multiple sources on the blockchain before letting people open files. We tested it in a few different situations.

a) Experimental Setup:

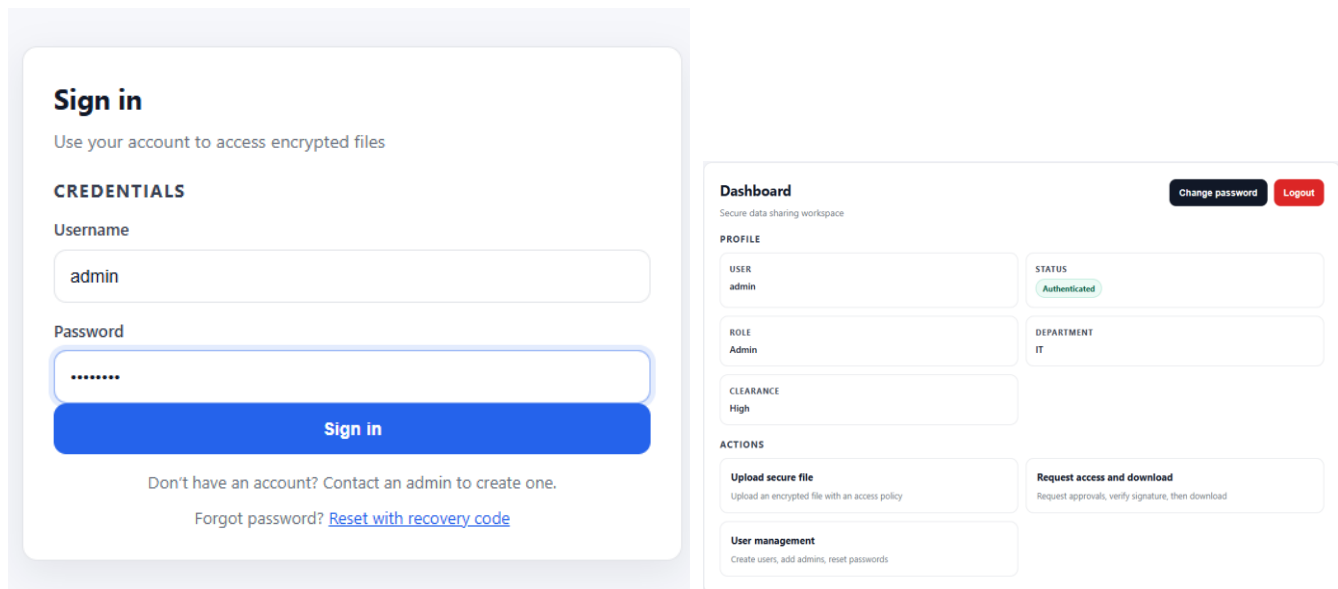
- We ran the tests on a normal computer with an Intel Core i5 processor, 16 GB of RAM, and Windows. We used Python 3.10. The behind-the-scenes stuff ran locally, and we used MongoDB Atlas (GridFS) to store the encrypted files.
- We made up a bunch of fake users with different roles, departments, and security levels to act like a real company. An admin uploaded some encrypted files with different access rules to see if the system could apply the rules correctly.
- We acted out different access situations, like when someone was allowed to access a file, when they broke the rules, when they didn't have enough blockchain approvals, and when they weren't allowed to access a file at all. We did each situation several times to make sure things were consistent.
- We checked if the encryption and decryption were right, if the access rules were correct, if the approval limits worked, and how fast the system was when uploading, requesting, and downloading files.
- We looked at the system logs to confirm that all the important stuff, like logins, access requests, rule decisions, approvals, and downloads, were recorded correctly for checking later.
- We did a bunch of test runs with different user details and approval answers to see if the system was stable and to make sure that people who weren't allowed to access files were always blocked.

Discussion: The test results showed that our system is a dependable and safe way to share data in a controlled way. The AES-256 file encryption kept things private, since the actual files were never stored openly. The access rules worked properly, stopping people without the needed details from getting to the approval part.

- The blockchain-based approval system stopped people from making access decisions on their own. Access was only given when enough approvals were in, making things open and preventing misuse. The fact that blockchain records can't be changed also made auditing better.
- The time it took to check the rules and approvals was okay for a secure data sharing platform. Even though blockchain added a bit of extra time, it didn't really slow things down too much and was worth it for the added security and accountability.
- Since we did the tests in a controlled place, performance might be different when used on a bigger scale or with many users. We could improve things by making blockchain interactions faster, making rule checking more efficient, and making the system work in different setups.

CHAPTER 11

RESULTS AND ANALYSIS



The image displays two side-by-side web interface components. The left component is a 'Sign in' form with the title 'Sign in' and a subtitle 'Use your account to access encrypted files'. It features a 'CREDENTIALS' section with a 'Username' field containing 'admin' and a 'Password' field with masked characters. A blue 'Sign in' button is at the bottom. Below the button, there is a link for users without an account and a link for password recovery. The right component is a 'Dashboard' for an 'Admin' user. It includes a 'Change password' button and a 'Logout' button. The dashboard shows user profile information: USER (admin), STATUS (Authenticated), ROLE (Admin), DEPARTMENT (IT), and CLEARANCE (High). It also lists actions: 'Upload secure file' (with a description), 'Request access and download' (with a description), and 'User management' (with a description).

Sign in
Use your account to access encrypted files

CREDENTIALS

Username
admin

Password

Sign in

Don't have an account? Contact an admin to create one.
Forgot password? [Reset with recovery code](#)

Dashboard
Secure data sharing workspace

Change password **Logout**

PROFILE

USER
admin

STATUS
Authenticated

ROLE
Admin

DEPARTMENT
IT

CLEARANCE
High

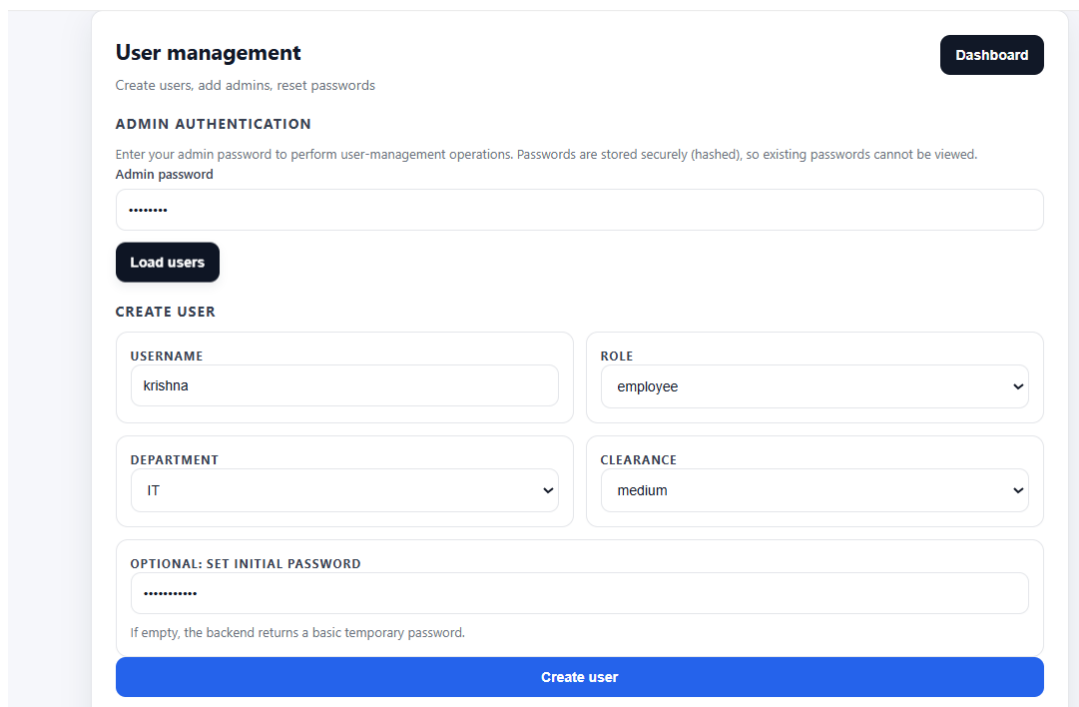
ACTIONS

Upload secure file
Upload an encrypted file with an access policy

Request access and download
Request approvals, verify signature, then download

User management
Create users, add admins, reset passwords

Figure 11.1: Admin Login and User Dashboard.



The image shows a 'User management' interface. It has a 'Dashboard' button in the top right. The main heading is 'User management' with a subtitle 'Create users, add admins, reset passwords'. Below this is a section for 'ADMIN AUTHENTICATION' with a note about password security and an 'Admin password' field. A 'Load users' button is present. The 'CREATE USER' section contains four input fields: 'USERNAME' (krishna), 'ROLE' (employee), 'DEPARTMENT' (IT), and 'CLEARANCE' (medium). There is an 'OPTIONAL: SET INITIAL PASSWORD' field. A 'Create user' button is at the bottom.

User management **Dashboard**

Create users, add admins, reset passwords

ADMIN AUTHENTICATION

Enter your admin password to perform user-management operations. Passwords are stored securely (hashed), so existing passwords cannot be viewed.

Admin password

Load users

CREATE USER

USERNAME
krishna

ROLE
employee

DEPARTMENT
IT

CLEARANCE
medium

OPTIONAL: SET INITIAL PASSWORD

If empty, the backend returns a basic temporary password.

Create user

Fig. 11.2: User Registration by Administrator.

Upload

Encrypt and upload files with an access policy

FILE

Select file

2025-POL-ODA-001-Evaluation_Policy_2025-26.pdf

Selected: 2025-POL-ODA-001-Evaluation_Policy_2025-26.pdf

WHO CAN ACCESS?

Pick one or more values in each category.

ROLES

☐ Admin

☐ Manager

☐ Accountant

☒ Employee

☐ Worker

DEPARTMENTS

☒ IT

☐ Finance

☐ HR

☐ Operations

CLEARANCE

☐ High

☒ Medium

☐ Low

GENERATED POLICY

This upload creates a policy like:
(role:employee) AND (dept:IT) AND (clearance:medium)

Fig. 11.3: Secure File Upload with Access Policy Definition.

CLUSTERS (1)

Search clusters

- Cluster0
 - admin
 - local
 - sample_mflix
 - secure_data_sharing
 - encrypted_files.chunks
 - encrypted_files.files

Generate query + Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT CODE

25 1 - 19 of 19

```

1  _id: ObjectId('699367902d2fd12dff3ab37b')
2  files_id: 6993678f2d2fd12dff3ab37a
3  n: 0
4  data: Binary.createFromBase64('EuVv7BePo9pX/99wvgDep45+yRuvf8mMud6Z1LNvzkQnqsRXQ0LW6LvpI2byuxT0FStLMkpK/w4SAmQjJLUGYwPrCOypSNAVB...')

```

CANCEL UPDATE

```

_id: ObjectId('699367912d2fd12dff3ab37c')
files_id: ObjectId('6993678f2d2fd12dff3ab37a')
n: 1
data: Binary.createFromBase64('6l9hw1p1L9kN5umihtxET7TdmQ1Byp80f0Ujter80x82bEXHqSwarWGRf+ihNCTxSc/f55Rem0TCBTCRp+d1w4wJLg3IdzN0YbL...', 0)

```

6993678f2d2fd12dff3ab37a

System Status: All Good

```

{
  "_id": {
    "$oid": "699367902d2fd12dff3ab37b"
  },
  "files_id": {
    "$oid": "6993678f2d2fd12dff3ab37a"
  },
  "n": 0,
  "data": {
    "$binary": {
      "base64": "EuVv7BePo9pX/99wvgDep45+yRuvf8mMud6Z1LNvzkQnqsRXQ0LW6LvpI2byuxT0FStLMkpK/w4SAmQjJLUGYwPrCOypSNAVBh5sq31RCyL8fTWtsbcFv2/Ix+VPHcnEtJ5/srIVuLEPjN9KVK8e1QPN/5m9sz7P/Ta53ps0eYr/oxvc3PIIXWkgTWY23rqJ4ryps0+x2BV+7LAHWBqyCTliQjTOz5Tuxtb12K8C8iGC6GX5DUiXV7Ud+CTG1MDfYsEEYNhQ15ZtWY9Xch5zKWHUpxEentBTP0WdbBHmmsV73U2YXmJolCq4349ZExSSiWnm1eR8ohD7IunEekuW1RCuWCrSONh1XfFNh/EYDbLttj0kqBQkL/2egTRG1VZN7A60FdPproZgzVJen1Uo6adpiPS6jYdLTmEX+2DRbkCHyntOEQLEcxpPBNXO11xE7142obANY4m97VinA9zeFoB/D2KS27RyyEkmuYkLwcaIFyQxcr54/Byf/mjX4baXz65BLomuvMdrW7nhOKUQozSudxFAOUoMYvsbw7hi5poB1NO6v6EKT/uElwcV0i3UwSxdddgzS7ZTtYPkd5TTiQse0bsBUjx29YMq0QG0Xh00APMIMGGE2JdgFLiSWUzrbwntprf3Xize6edJ5I5MB87xqF/bT8F10CVtk1nh1LiKTimecON/oi+ekUnFdh3ZwXgFAVkgG5OCwf1mN4Hh+VJbZ10BhJ94Jdb3OnE4V744sLZ8pn0a/gSkk6oAHbqcPwWN3M2zKn0huJ4GuE2rByigIdyDkeu15"
    }
  }
}

```

Fig. 11.4: Encrypted File Storage Confirmation.

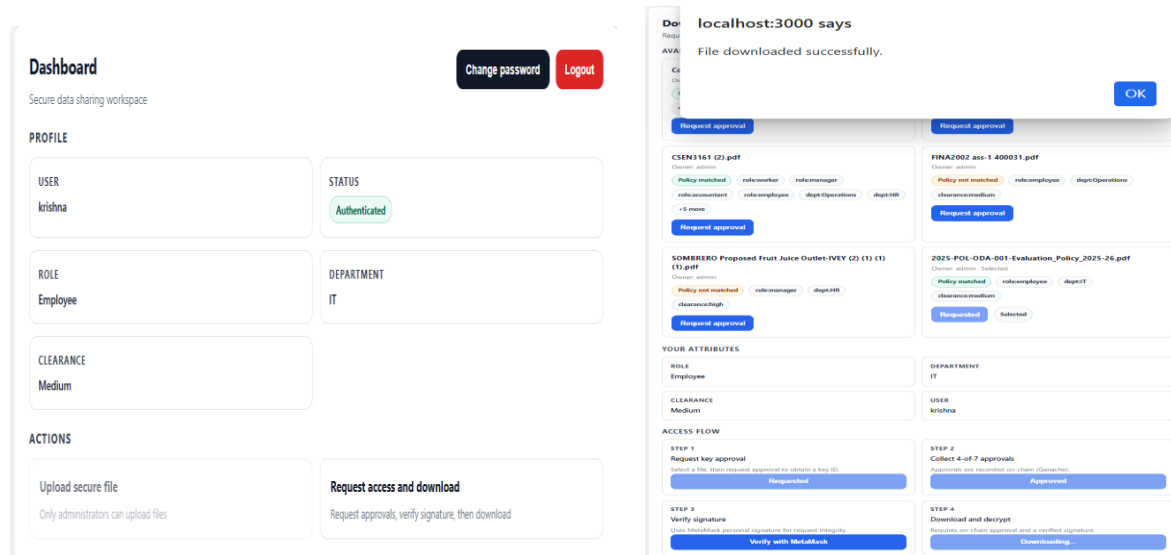


Fig. 11.5: User Login and File Access Request.

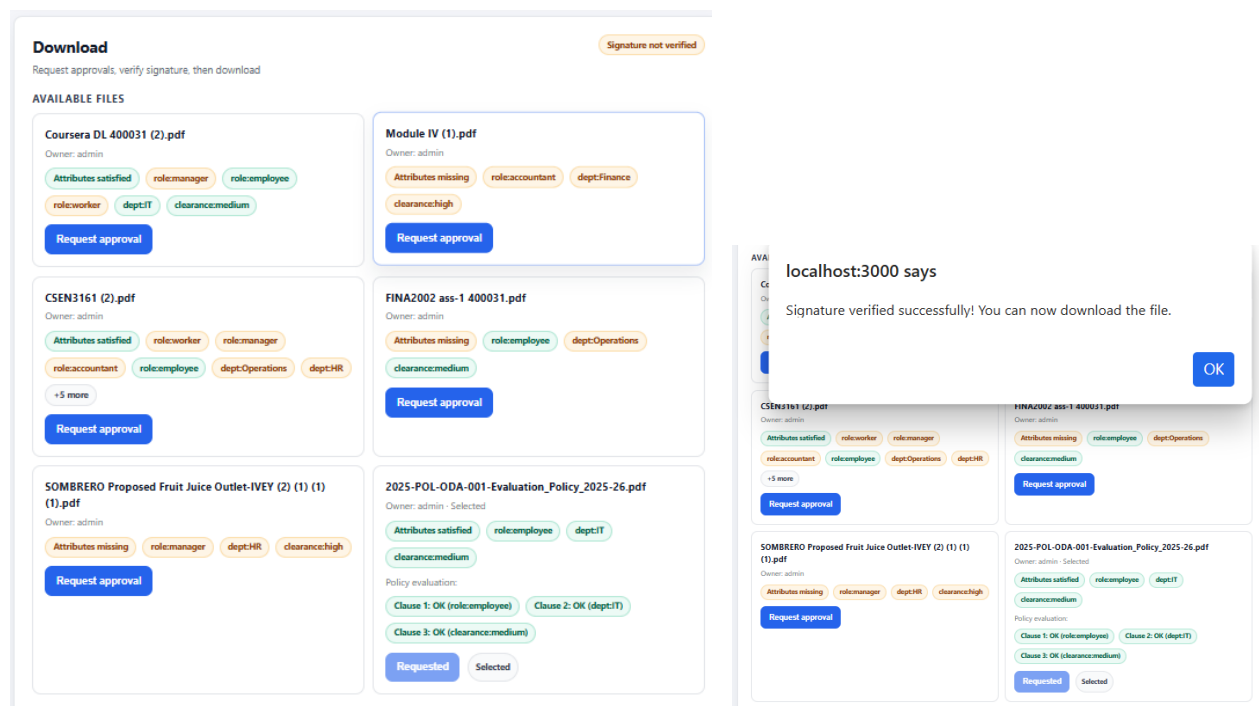


Fig. 11.6: Access Policy Verification Result.

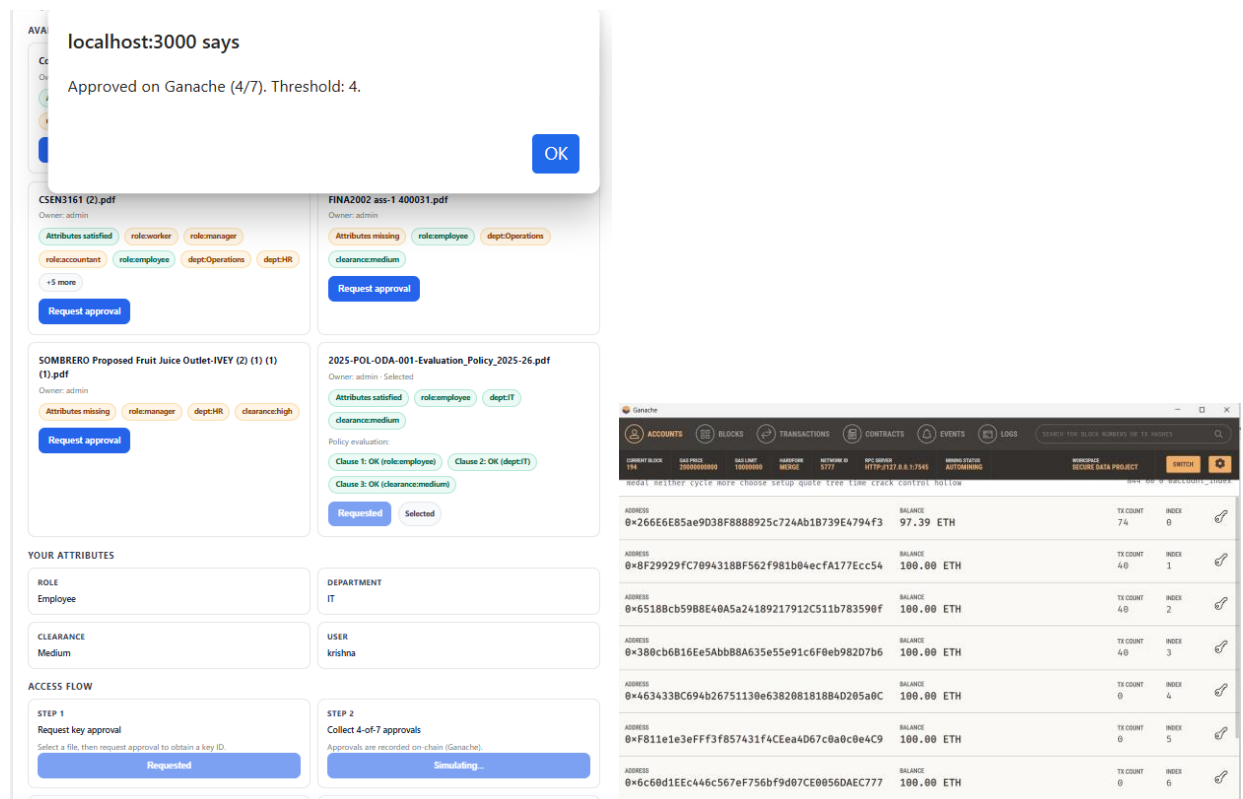


Fig. 11.7: Blockchain-Based Multi-Authority Approval.

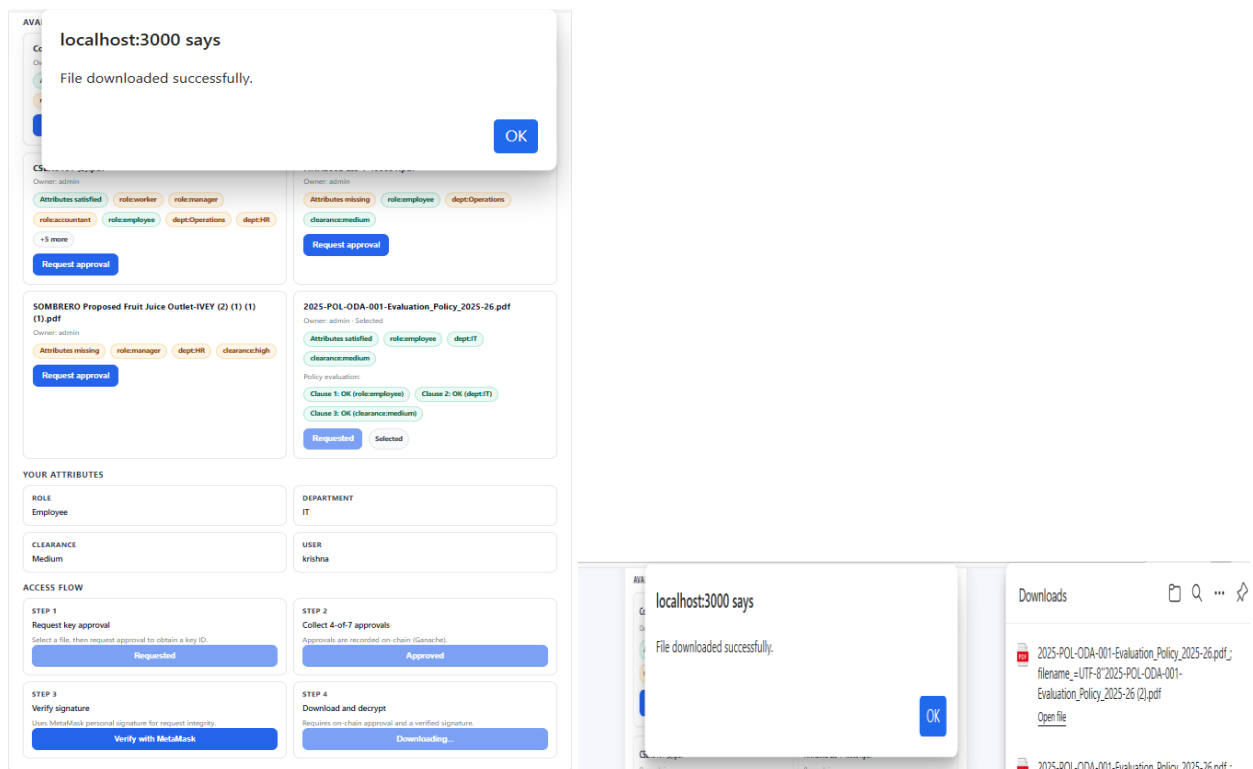


Fig. 11.8: Secure File Download After Approval.

CHAPTER 12

CONCLUSION

This project made a secure system for sharing data that tackles major issues with keeping data private, controlling who gets to see it, and managing permissions. Because companies are using cloud storage and spread-out systems more and more, old ways of controlling access often don't do enough to stop people who aren't allowed in or employees who misuse their access. This new system fixes those problems by using several layers of security. Files are kept private using AES-256 encryption, which means they're encrypted before they're stored, so the original data is never just sitting there. This makes sure they're well-protected, even if the storage gets hacked. Access is carefully managed using rules that look at things like a user's role, department, and security level. This lets the system make access decisions that fit the company's actual needs. To make permissions even stronger, the system uses a multi-person approval method powered by blockchain. Access to important files is only given if enough people approve it, which makes things clear, keeps people responsible, and stops one person from making access decisions on their own. Also, every access is logged, which helps with checking who did what and tracking down problems. All in all, this project shows a useful solution for sharing data safely by mixing encryption, rule-based access control, and permission from different people. The design is a good starting point for making things even better and using it in real situations.

CHAPTER 13

FUTURE WORK

- Switch from a local blockchain (like Ganache) to a real-world testnet or mainnet when you're ready to go live.
- Use full cryptographic CP-ABE instead of just basic policy rules.
- Include AES-GCM for built-in data protection and security.
- Use JWT/OAuth2 for better login and session control.
- Create dashboards for different roles and send out real-time alerts for approvals.
- Include ways to cancel keys and update policies on the fly.
- Swap out SQLite for PostgreSQL to handle more users and activity.
- Put the system in a containerized setup on the cloud (like Docker/Kubernetes).
- Add detailed logs, security monitoring, and rate limits.
- Build a mobile app for secure access and approvals.

CO-PO-PSO Mapping

CO–PO–PSO Articulation Table (Project Title: Secure Data Sharing in Cloud Using Attribute-Based Encryption (ABE))

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	2	2	2	2	1	1	2	2	3	2	1	2	2	2
CO2	3	3	3	2	3	1	2	2	2	2	2	1	3	3	2
CO3	3	3	3	3	3	1	2	2	2	2	2	1	3	3	2
CO4	3	3	2	3	2	1	1	2	2	2	2	1	3	2	2
CO5	2	2	2	2	1	2	2	2	2	3	2	2	2	1	3
CO6	2	2	2	2	2	1	1	3	2	3	3	2	2	2	3

CO–PO–PSO Detailed Justification Tables (Project Title: Secure Data Sharing in Cloud Using Attribute-Based Encryption (ABE))

CO1: Demonstrate the ability to present and explain the complete project work effectively through professional demonstrations and presentations.

PO / PSO	Level	Justification
PO1	3	Strong engineering knowledge was demonstrated while presenting AES-256 encryption, blockchain smart contracts, and attribute-based access control concepts.
PO2	2	Problem analysis was explained during presentation by identifying limitations of centralized cloud storage systems.
PO3	2	The system design including secure architecture and decentralized approval mechanism was presented clearly.
PO4	2	Investigation results such as encryption, access verification, and blockchain approval were demonstrated.
PO5	2	Modern tools like FastAPI, React, Solidity, and Ganache were explained during demonstration.
PO6	1	Societal relevance such as secure data sharing in healthcare and banking was briefly discussed.
PO7	1	Sustainability considerations like secure long-term data protection were moderately explained.
PO8	2	Ethical responsibility in protecting sensitive data was communicated.
PO9	2	Team collaboration was demonstrated during group presentation.

PO10	3	Strong communication skills were used to present technical concepts effectively.
PO11	2	Project planning and management were explained during presentation.
PO12	1	Learning new technologies like blockchain was demonstrated.
PSO1	2	Programming knowledge in Python and system implementation was presented.
PSO2	2	Application development using cloud and blockchain was explained.
PSO3	2	Ethical and secure system development was demonstrated.

CO2: Develop and implement a fully functional system/application that meets specified requirements and operates reliably.

PO / PSO	Level	Justification
PO1	3	Engineering knowledge was applied in implementing encryption and blockchain-based access control.
PO2	3	Problem of insecure cloud sharing was analyzed and solved through decentralized secure system.
PO3	3	Problem of insecure cloud sharing was analyzed and solved through decentralized secure system.
PO4	2	System behavior was analyzed and verified during implementation.
PO5	3	Modern tools such as FastAPI, React, Solidity, and cryptographic libraries were used extensively.
PO6	1	The system contributes to secure data sharing for society.
PO7	2	Secure cloud solutions help sustainable digital infrastructure.
PO8	2	Ethical handling of sensitive data was implemented.
PO9	2	Team members worked collaboratively in development.
PO10	2	Technical documentation and reporting were prepared.
PO11	2	Project planning and development management were followed.
PO12	3	New technologies were learned during implementation.
PSO1	3	Programming skills in Python and cryptography were applied.
PSO2	3	Cloud-based secure application was developed.
PSO3	2	Secure and ethical computing practices were followed.

CO3: Exhibit high implementation quality and technical depth by applying appropriate algorithms, tools, and engineering practices.

PO / PSO	Level	Justification
PO1	3	Strong technical knowledge was applied in AES encryption and CP-ABE logic.
PO2	3	Security challenges were analyzed and solved using cryptographic techniques.
PO3	3	Efficient and secure system architecture was designed.
PO4	3	Research-based encryption and blockchain methods were implemented.
PO5	3	Advanced tools like blockchain and cryptography libraries were used.
PO6	1	The system ensures secure data handling for society.
PO7	2	Secure storage contributes to sustainable digital systems.
PO8	2	Ethical security practices were implemented.
PO9	2	Team collaboration ensured high quality implementation.
PO10	2	Technical documentation was prepared properly.
PO11	2	Engineering management practices were followed.
PO12	1	Learning modern cryptography enhanced lifelong learning.
PSO1	3	Advanced programming and algorithm implementation were applied.
PSO2	3	Secure cloud application development was performed.
PSO3	2	Ethical engineering practices were applied.

CO4: Perform systematic testing and verification to ensure the correctness, reliability, and performance of the developed system.

PO / PSO	Level	Justification
PO1	3	Engineering principles were used in testing encryption and system reliability.
PO2	3	Problems during testing were analyzed and resolved.
PO3	2	Design improvements were made based on testing results.
PO4	3	System testing and validation were performed.
PO5	2	Testing tools and development frameworks were used.
PO6	1	System ensures reliable secure data sharing.
PO7	1	Reliable systems contribute to sustainability.

PO8	2	Ethical testing ensured secure access.
PO9	2	Team performed collaborative testing.
PO10	2	Test results were documented clearly.
PO11	2	Testing process was managed effectively.
PO12	1	Testing process was managed effectively.
PSO1	3	Programming skills were applied in testing modules.
PSO2	2	Cloud-based application testing was performed.
PSO3	2	Ethical testing practices were followed.

CO5: Analyze project outcomes, draw meaningful conclusions, and engage in critical discussion for further improvement and future scope.

PO / PSO	Level	Justification
PO1	2	Engineering knowledge was used to evaluate system performance.
PO2	2	Security outcomes were analyzed critically.
PO3	2	Improvements were suggested for system design.
PO4	2	Performance evaluation was conducted.
PO5	1	Tools were used for analysis.
PO6	2	Secure systems benefit society.
PO7	2	Secure cloud systems contribute to sustainability.
PO8	2	Ethical implications were considered.
PO9	2	Team discussions were conducted.
PO10	3	Conclusions were clearly documented.
PO11	2	Future project planning was discussed.
PO12	2	Continuous learning was demonstrated.
PSO1	2	Programming system performance was evaluated.
PSO2	1	Application improvements were suggested.
PSO3	3	Ethical and secure improvements were proposed.

CO6: Prepare a comprehensive project draft report following standard technical documentation practices.

PO / PSO	Level	Justification
PO1	2	Engineering concepts were documented clearly.
PO2	2	Problem analysis was included in report.
PO3	2	System design was documented.
PO4	2	Testing and results were documented.
PO5	2	Tools and technologies were explained.
PO6	1	Social impact was included.
PO7	1	Sustainability considerations were documented.
PO8	3	Ethical data handling was documented clearly.
PO9	2	Team contribution was documented.
PO10	3	Professional report writing was demonstrated.
PO11	3	Project management was documented.
PO12	2	Learning outcomes were included.
PSO1	2	Programming implementation was documented.
PSO2	2	Cloud application design was documented.
PSO3	3	Cloud application design was documented.

Sustainable Development Goals (SDGs)



CO–SDG Mapping with Justification (Project Title: Secure Data Sharing in Cloud Using Attribute-Based Encryption (ABE))

SDG No.	SDG Title	Why it Strongly Applies
SDG 4	Quality Education	Ensures secure storage and sharing of academic data.
SDG 9	Industry, Innovation, and Infrastructure	Develops secure cloud infrastructure using encryption and blockchain.
SDG 16	Peace, Justice and Strong Institutions	Provides secure, transparent, and tamper-proof data access.

COs	SDG No(s)	SDG Title(s)
CO1	4, 9	Quality Education; Industry, Innovation and Infrastructure.
CO2	9, 16	Industry, Innovation and Infrastructure; Peace, Justice and Strong Institutions
CO3	9, 16	Industry, Innovation and Infrastructure; Peace, Justice and Strong Institutions
CO4	9, 16	Industry, Innovation and Infrastructure; Peace, Justice and Strong Institutions
CO5	9, 16	Industry, Innovation and Infrastructure; Peace, Justice and Strong Institutions
CO6	4, 16	Quality Education; Peace, Justice and strong institutions.

CO-wise SDG Mapping with Justification

CO1: Demonstrate the ability to present and explain the complete project work effectively through professional demonstrations and presentations.

Mapped SDGs:

- SDG 9 – Industry, Innovation and Infrastructure
- SDG 16 – Peace, Justice and Strong Institutions

Justification:

- Clear presentation of secure cloud architecture and encryption techniques contributes to knowledge sharing in cybersecurity and secure cloud systems.
- Demonstrations promote awareness of secure and transparent data sharing solutions for organizations and institutions.

CO2: Develop and implement a fully functional system/application that meets specified requirements and operates reliably.

Mapped SDGs:

- SDG 9 – Industry, Innovation and Infrastructure
- SDG 16 – Peace, Justice and Strong Institutions

Justification:

- The system strengthens secure digital infrastructure using encryption and blockchain-based access control.
- Reliable authentication and decentralized approval improve security, transparency, and trust in institutional data sharing.

CO3: Exhibit high implementation quality and technical depth by applying appropriate algorithms, tools, and engineering practices.

Mapped SDGs:

- SDG 9 – Industry, Innovation and Infrastructure
- SDG 16 – Peace, Justice and Strong Institutions

Justification:

- Advanced encryption algorithms and blockchain technologies improve cybersecurity infrastructure.
- Secure data sharing protects business and organizational information, supporting safe and reliable digital operations.

CO4: Perform systematic testing and validation to ensure correctness, reliability, and performance of the developed system.

Mapped SDGs:

- SDG 16 – Peace, Justice and Strong Institutions.

- SDG 9 – Industry, Innovation and Infrastructure

Justification:

- Reliable testing ensures secure and trustworthy institutional data sharing systems.
- Validation improves the performance and reliability of secure digital infrastructure.

CO5: Analyze project outcomes, draw meaningful conclusions, and engage in critical discussion for further improvement and future scope.

Mapped SDGs:

- SDG 9 – Industry, Innovation and Infrastructure
- SDG 16 – Peace, Justice and Strong Institutions.

Justification:

- Outcome analysis supports continuous improvement of Secure Cloud Infrastructure and Cybersecurity systems.
- Enhancements contribute to more secure, transparent, and resilient data sharing environments.

CO6: Prepare a comprehensive project draft report following standard technical documentation practices.

Mapped SDGs:

- SDG 9 – Industry, Innovation and Infrastructure

Justification:

- Well-structured documentation supports research and development in secure cloud computing.
- Proper reporting promotes collaboration and knowledge sharing in cybersecurity and secure system development.

REFERENCES

- [1] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-Policy Attribute-Based Encryption," in Proc. IEEE Symposium on Security and Privacy (S&P), (2007), pp. 321–334.
- [2] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-Based Encryption for Fine-Grained Access Control of Encrypted Data," in Proc. ACM CCS, (2006), pp. 89–98.
- [3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System,"(2008).[Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [4] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Project Yellow Paper, (2014).
- [5] D. Thilakanathan, S. Chen, S. Nepal, R. Calvo, and L. Alem, "A Platform for Secure Monitoring and Sharing of Generic Health Data in the Cloud," Future Generation Computer Systems, vol. 35, pp. 102–113, (2014).
- [6] Y.-W. Hwang, I.-Y. Lee, "An SKP-ABE Scheme for Secure and Efficient Data Sharing in Cloud Environments," IEEE Access, vol. 10, pp. 40894–40907, (2022).
- [7] I. El Ghoubach, R. Mishra, and K. Aghababaei, "FP2E: Flexible, Effective and Privacy-Preserving Cloud Data Sharing Scheme," in Proc. IEEE International Conference on Cloud Computing, (2016), pp. 784–789.
- [8] R. Fatima and M. Naveeduddin, "A Comprehensive Survey of Attribute-Based Encryption Techniques in Cloud Computing," Journal of King Saud University – Computer and Information Sciences, vol. 37, no. 1, pp. 1–22, (2025).
- [9] M. Zhao, H. Ma, and J. Qi, "Lattice-Based ABE with Multi-Ciphertext Equality Test in Cloud Computing," Journal of King Saud University – Computer and Information Sciences, vol. 37, no. 2, pp. 1–15, (2025).
- [10] A. Beimel, "Secure Schemes for Secret Sharing and Key Distribution," Ph.D. dissertation, Israel Institute of Technology, Technion, (1996).
- [11] A. Shamir, "How to Share a Secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, (1979).
- [12] NIST, "Advanced Encryption Standard (AES)," FIPS Publication 197, National Institute of Standards and Technology, (2001).
- [13] D. Johnson, A. Menezes, and S. Vanstone, "The Elliptic Curve Digital Signature Algorithm (ECDSA)," International Journal of Information Security, vol. 1, no. 1, pp. 36–63, (2001).
- [14] S. Wang, Y. Zhang, and Y. Zhang, "A Blockchain-Based Fine-Grained Data Sharing Scheme for E-Health Systems in Smart Cities," Future Generation Computer Systems, vol. 81, pp. 420–429, (2018).
- [15] C. Dwork, "Differential Privacy," in Proc. International Colloquium on Automata, Languages and Programming (ICALP), (2006), pp. 1–12.

- [16] L. A. Dunning and R. Kresman, "Privacy Preserving Data Sharing with Anonymous ID Assignment," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 2, pp. 402–413, (2013).
- [17] M. Ali, S. U. Khan, and A. V. Vasilakos, "Security in Cloud Computing: Opportunities and Challenges," *Information Sciences*, vol. 305, pp. 357–383, (2015).
- [18] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," in *Proc. IEEE International Congress on Big Data*, (2017), pp. 557–564.
- [19] A. Lewko and B. Waters, "Decentralizing Attribute-Based Encryption," in *Proc. Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, (2011), pp. 568–588.
- [20] R. Canetti and S. Goldwasser, "An Efficient Threshold Public Key Cryptosystem Secure Against Adaptive Chosen Ciphertext Attack," in *Proc. EUROCRYPT*, (1999), pp. 90–106.



Bengaluru

NH 207, Nagadenehali,
Doddaballapur Taluk,
Bengaluru - 561203,
Karnataka, India

Hyderabad

Rudraram, Patancheru
mandal, Hyderabad -
502329, Telengana, India

Visakhapatnam

Gandhi Nagar, Rushikonda,
Visakhapatnam- 530045,
Andhra Pradesh, India



Bengaluru



Visakhapatnam

GIMSR

Medical College Rd,
Nagar, Rushikonda,
Visakhapatnam - 530045
Pradesh, India

Bengaluru City Office

No 5/1, First Floor, Prestige
Terraces, Union Street,
Infantry Road, Bengaluru -
560003 Karnataka, India

Hyderabad City Office

6th floor, TP House, Cyber Gandhi
Hills Colony, VIP Hills,
Jaihind Enclave, Madhapur, Andhra
Hyderabad - 500081,
Telangana, India



Hyderabad



GIMSR
Visakhapatnam